

Trellix Final Report

Author: Issa El Mokadem

Program: 2º DAW – I.E.S. Abdera

Project Type: Full-Stack Web Application (Task Board)

1. Project Summary

Trellix is a Kanban-style task management web application inspired by Trello. Built as the final project for the 2º DAW program, it demonstrates full-stack development capabilities across frontend, backend, database, and DevOps.

Tech Stack: React + Vite + Tailwind CSS (frontend) | FastAPI + SQLAlchemy (backend) | MariaDB (database) | Docker + Nginx (deployment)

Core Features:

- User authentication with JWT and role-based access control (Admin/User)
 - Board, list, and card CRUD operations
 - Drag-and-drop task movement with optimistic UI
 - Weather sidebar widget via open-meteo API
 - Admin panel for user management (block/unblock/delete)
-

2. Task Tracking (Jira)

2.1 Overview

All development work was tracked in Jira using a Scrum board with the following issue types: Epic, Story, Feature, Task, and Bug.

Metric	Count
Total issues	95
Features, tasks, and stories	89
Bugs	6
Completed	94
Pending	1

2.2 Task Distribution by Phase

The project followed vertical slicing: feasibility and planning first, then design, followed by implementation in layers (auth → boards/lists → cards/drag-and-drop → admin → polish).

Phase	Key Deliverables	Status
Planning & Feasibility	Technology selection, feasibility study, project roadmap	✅ Complete
Design	User stories, ER diagram, class diagram, UI prototypes (Figma), component architecture	✅ Complete
Backend Core	FastAPI project structure, MariaDB + ORM, JWT auth, RBAC	✅ Complete
CRUD Operations	Boards, lists, cards endpoints with full test coverage	✅ Complete
Frontend	React component tree, Zustand store, dnd-kit integration, Tailwind design system	✅ Complete
DevOps	Docker Compose (4 containers), Nginx reverse proxy, development environment	✅ Complete
Documentation & Delivery	Manuals, incident procedure, final report (this document)	🔄 In Progress

2.3 Pending Work

Issue	Description	Reason
SCRUM-93	Complete minimal tests coverage	Non-blocking — core features are tested; edge cases and

No other pending tasks. All 89 features, stories, and tasks reached Done status.

3. Incident Management

3.1 Bug Summary

A total of **6 bugs** were registered during development. **5 have been resolved and closed**; 1 remains open as low-priority tech debt.

ID	Summary	Priority	Status	Impact
SCRUM-94	Privilege escalation via PATCH /me endpoint	Medium	✓ Closed	Security — users could modify their own role/is_active fields
SCRUM-104	Application crash when JWT token expires	Medium	✓ Closed	UX — unhandled promise rejection caused white screen
SCRUM-109	Login rejected valid emails (backend only checked username)	Medium	✓ Closed	Functional — users unable to log in with email
SCRUM-111	Delete modal persisted after deleting a card	Medium	✓ Closed	UX — stale UI state after delete operation
SCRUM-112	Database init incompatible with multi-replica deployment	Medium	✓ Closed	DevOps — lazy init strategy assumed single instance
SCRUM-93	Incomplete test coverage	Medium	☐ Open	Quality — admin and edge-case

				tests not written
--	--	--	--	-------------------

3.2 Most Significant Incidents

SCRUM-94 — Security: Privilege Escalation

Users could modify their own `role` and `is_active` fields via the profile update endpoint. Root cause: the generic `UserUpdate` schema exposed admin-only fields. Fixed by creating a dedicated `UserMeUpdate` schema that excludes sensitive fields. Security regression tests added.

SCRUM-104 — Stability: Token Expiry Crash

When the JWT token expired, the frontend did not handle the 401 response cleanly, causing an unhandled promise rejection that crashed the UI. Fixed by adding an Axios response interceptor that redirects to `/login` on 401 errors.

SCRUM-109 — Functional: Email Login Broken

The login page placeholder text said "username or email", but the backend only queried by username. Users attempting email login received "Incorrect username or password". Fixed by expanding the query to check both fields.

3.3 Lessons Learned

- **Schema design matters:** always use dedicated input schemas for sensitive endpoints (lesson from SCRUM-94)
- **Error boundaries are essential:** every API call should have a handled failure path (lesson from SCRUM-104)
- **Frontend-backend contract alignment:** placeholder text and actual behavior must match (lesson from SCRUM-109)

4. Final Assessment

4.1 Objective Compliance

Objective	Status	Evidence
Full CRUD for boards, lists, cards	✓ Met	FastAPI endpoints + React UI

JWT authentication + RBAC	✓ Met	AuthContext + protected routes + admin panel
Drag-and-drop task management	✓ Met	dnd-kit with optimistic UI + PATCH /cards/{id}/move
External API integration	✓ Met	WeatherWidget via open-meteo
Docker containerization	✓ Met	4 containers: nginx, backend, frontend, mariadb
Responsive design	✓ Met	Tailwind CSS, tested across viewports

4.2 Quality Assessment

Strengths:

- Clean separation of concerns: React components are presentational, logic lives in Zustand stores and FastAPI endpoints
- Optimistic UI for drag-and-drop provides instant feedback with server-side persistence and rollback
- Backend test coverage for core endpoints (auth, CRUD, card movement)
- Consistent design language ("Digital Curator" theme) applied throughout

Areas for Improvement:

- Frontend test coverage is limited — E2E tests for critical flows would increase confidence
- Board-level permissions (private boards) were identified in user interviews but deprioritized
- Due dates on cards were planned but not implemented in the current iteration
- Admin panel works but lacks batch operations and search/filter capabilities

4.3 Improvement Proposals

1. **Add Playwright E2E tests** for the core user flows: register → create board → add card → drag to another list
2. **Implement board sharing** with member-level permissions (read-only, edit, admin)
3. **Add due dates and reminders** as described in the original feature plan (F-004)
4. **Upgrade weather integration** from sidebar widget to per-card context (as originally envisioned)

5. **Add notification system** using WebSockets for real-time collaboration cues

5. Conclusion

Trellix successfully delivers a functional, well-architected Kanban board application. All core objectives defined in the feasibility study and user stories have been met. The project demonstrates competence across the full web development stack — from database design and REST API implementation to modern React UI development and containerized deployment.

The systematic use of Jira for task tracking and incident management provided traceability throughout the development cycle. With 94 of 95 issues resolved and only minor test coverage gaps remaining, the project meets the quality bar expected for a DAW final project.

The Improvement Proposals in Section 4.3 outline a natural roadmap for future iterations, confirming that the architecture is designed for extensibility beyond the current scope.